Fase 2: Ejecución

1. Introducción y Objetivo

La clasificación de imágenes es una de las actividades principales de la visión computacional. Consiste en asignar de forma automática una etiqueta a una imagen. En este proyecto, se plantea y resuelve un problema de clasificación de imágenes usando una red neuronal convolucional (CNN) construida con Keras y TensorFlow en Python.

El problema elegido es poder identificar la variedad de un grano de arroz clasificando fotografías. La verificación de la variedad del grano de arroz es relevante y aplicable en el control de calidad y en la autenticidad de productos alimentarios. El objetivo es aplicar aprendizaje profundo a esta problemática recorriendo todas las fases de un proyecto de procesamiento de imágenes: obtención de datos, división, preprocesamiento, construcción y entrenamiento del modelo, evaluación y conclusión.

2. Identificación de la Tarea y Obtención de Datos

Este proyecto es sobre una clasificación multiclase de imágenes. Se va a dar una fotografía de un grano de arroz al modelo y se debe de predecir una de cinco variedades: Arborio, Basmati, Ipsala, Jasmine y Karacadag.

Se utilizó el *Rice Image Dataset* publicado en Kaggle por Murat Koklu (https://www.kaggle.com/datasets/muratkokludataset/rice-image-dataset). El conjunto proviene de la investigación de Koklu, Cinar y Taspinar sobre clasificación de variedades de arroz y es de acceso público. Contiene 75,000 imágenes a color en formato JPG de 250 x 250 píxeles. Cada imagen contiene un grano de arroz con un fondo negro uniforme.

La siguiente tabla muestra la composición del dataset y subconjunto utilizado:

Clase	Imágenes en el dataset	Imágenes usadas
Arborio	15,000	3,000
Basmati	15,000	3,000
Ipsala	15,000	3,000
Jasmine	15,000	3,000
Karacadag	15,000	3,000
Total	**75,000**	**15,000**

El dataset está perfectamente balanceado con 15,000 imágenes por clase. Para reducir el tiempo de entrenamiento, se tomó una muestra aleatoria de 3,000 imágenes por clase con semilla fija con un valor de 42. Al fijar la semilla, se mantiene el balance y el experimento es reproducible. Las imágenes corresponden a las mismas variedades que se desea identificar y fueron capturadas en condiciones homogéneas.

El proyecto se ejecutó en Google Colab con GPU. Se emplearon TensorFlow y Keras para el modelo y pipeline de datos. Además se utilizó scikit-learn para la partición y las métricas. Finalmente, se empleó matplotlib y seaborn para las gráficas. Las imágenes no contienen personas ni datos personales, por lo que no existen riesgos de privacidad. La principal consideración ética es la validez de los datos, porque los datos fueron tomados en laboratorio. Por ende, el desempeño reportado no debe extrapolarse a condiciones reales sin una validación adicional.

3. División de los Datos

El subconjunto de 15,000 imágenes se dividió de forma estratificada para conservar la misma proporción de clases en el entrenamiento, validación y prueba. El conjunto de prueba no se utilizó durante el entrenamiento ni para elegir hiperparámetros.

La siguiente tabla muestra la división estratificada de los datos. Por clase son 2,100 imágenes en entrenamiento, 450 en validación y 450 en prueba:

Conjunto	Proporción	Imágenes
Entrenamiento	70%	10,500
Validacion	15%	2,250
Prueba	15%	2,250

4. Preprocesamiento

El preprocesameinto se implementó dentro de un pipeline tf.data aplicado de forma identica a los tres conjuntos:

- **Formato y tamaño:** decodificación del JPG a tres canales y redimensionado a 128 x 128 píxeles.
- **Escalado:** los valores de píxel se convierten a punto flotante y se dividen en 255 resultando en el rango para los valores de [0,1]. Esto estabiliza y acelera el entrenamiento.
- **Escala de grises:** el pipeline incluye un valor booleano (GRAYSCALE) para convertir las imágenes a un canal (gris). El modelo principal conserva el color, porque el tono puede aportar información. Se hizo el entrenamiento completo con GRAYSCALE = True para medir su efecto.
- **Codificación de etiquetas:** las cinco clases se codifican en formato one-hot para que fuera compatible con la salida softmax y la pérdida de entropía cruzada categórica.
- **Eficiencia:** el conjunto de entrenamiento se mezcla en cada época, se usa lectura de datos en paralelo, lotes de 32 imágenes y prefetch.

Se validó que un lote (batch) tiene forma (32, 128, 128, 3) con valores entre 0 y 1 y etiquetas de forma (32, 5):

Valor	Significado
32	Tamaño del lote: la red procesa 32 imágenes a la vez en cada paso de entrenamiento.
128, 128	Alto y ancho de cada imagen en pixeles (128 x 128).
3	Canales de color: rojo, verde y azul (RGB). Una imagen escala de grises tendría 1.

Etiquetas de forma (32, 5), porque hay una etiqueta por imagen (32 imágenes) y cada una es un vector de 5 posiciones por el one-hot encoding.

5. Creación y Entrenamiento de la Red Neuronal Convolucional

Arquitectura

Se diseñó una CNN secuencial con tres bloques convolucionales que extraen características de complejidad creciente (bordes y contornos formas del grano rasgos globales) seguidos de un clasificador denso. Al inicio se incluye un aumento de datos (giros y rotación) que solamente está activo durante el entrenamiento y ayuda que el modelo no dependa de la orientación del grano.

La siguiente tabla muestra la arquitectura de la CNN para los 4,288,325 parámetros entrenables:

#	Capa	Configuracion	Salida
1	Entrada + Aumento de Datos	RandomFlip (horizontal y vertical), RandomRotation (0.1)	128 x 128 x 3
2	Conv2D + ReLU	32 filtros 3 x 3, padding = same	128 x 128 x 32
3	MaxPooling2D	2 x 2	64 x 64 x 32
4	Conv2D + ReLU	64 filtros 3 x 3, padding = same	64 x 64 x 64
5	MaxPooling2D	2 x 2	32 x 32 x 64
6	Conv2D + ReLU	128 filtros 3 x 3, padding = same	32 x 32 x 128
7	MaxPooling2D	2 x 2	16 x 16 x 128
8	Flatten		32,768
9	Dense + ReLU	128 neuronas	128
10	Dropout	tasa 0.5	128
11	Dense + Softmax	5 neuronas (una por variedad)	5

La mayor parte de los parámetros (4,194,432) se encuentran en la capa densa posterior a la capa de Flatten. Las tres capas convolucionales suman 93,248 y la capa de salida 645. Se utilizó Dropout de 0.5 como regularización para reducir el riesgo de sobreajuste.

Parámetros de Entrenamiento

La siguiente tabla muestra los hiperparámetros y configuración del entrenamiento:

Parámetro	Valor
Optimizador	Adam, Tasa de Aprendizaje 0.001
Función de Pérdida	Categorical Cross-Entropy
Métrica de Seguimiento	Accuracy (Precisión)
Tamaño de Lote	32

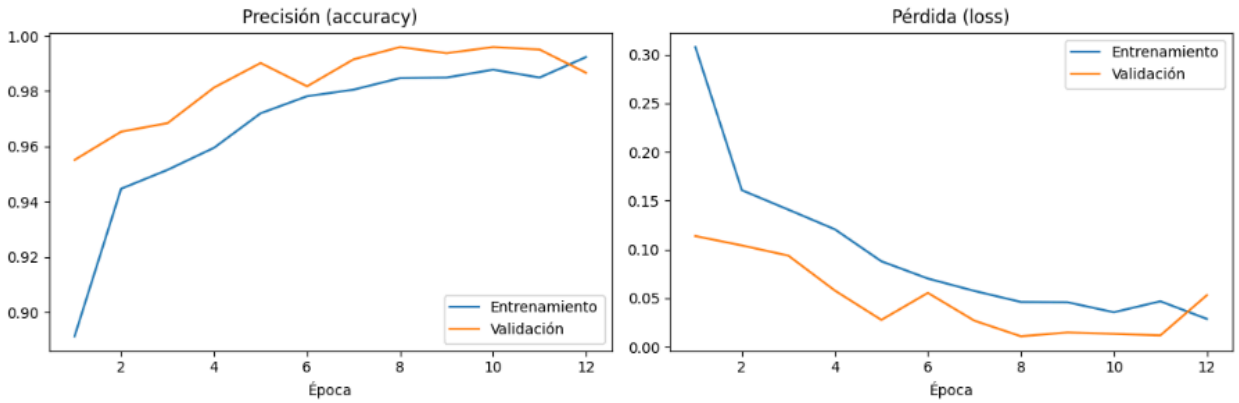
Épocas Máximas	15
Callbacks	EarlyStopping (monitor val_loss, paciencia 4, restaurando los mejores pesos) y ModelCheckpoint (guarda el mejor modelo)
Semilla Aleatoria	42

En la corrida del código usando el GRAYSCALE = FALSE (conservando el color original), el entrenamiento se detuvo automáticamente en la época 12 por EarlyStopping, porque la mejor pérdida de validación fue la de la época 8 y no mejoró durante las siguientes 4 épocas (las 4 épocas de paciencia). Los pesos de la época 8 se restauran antes de evaluar el modelo.

6. Evaluación de los Resultados

Curvas de Entrenamiento

La siguiente figura muestra la precisión y pérdida en entrenamiento y validación por época (modelo a color):



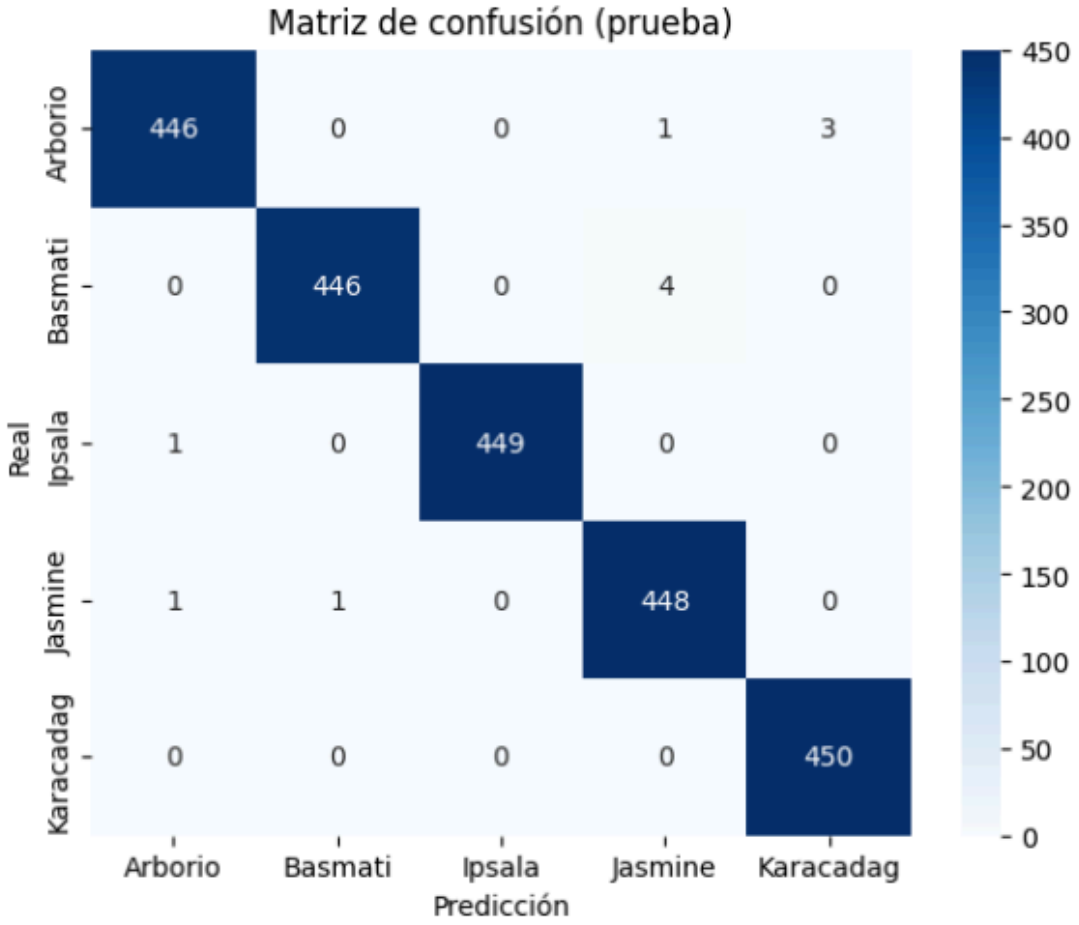
La precisión de entrenamiento sube de 0.891 en la primera época a 0.992 en la última época. Adicionalmente, la pérdida baja de 0.308 a 0.029. En validación, la precisión máxima de 0.996 y la pérdida mínima de 0.0109 se alcanzan en la época 8. Durante casi todo el entrenamiento la validación supera el entrenamiento, debido a que el Dropout y el aumento de datos solamente se activan durante la fase de entrenamiento.

La validación muestra dos oscilaciones en la precisión: una caída en la época 6 y otra en la época 12. Como la pérdida de validación no mejoró durante 4 etapas seguidas después de la época 8, EarlyStopping detuvo el entrenamiento en la época 12 y restauró los pesos del modelo a los pesos de la época 8. Con una sola época no se puede saber si el repunte de la 12 es inestabilidad del optimizador o el inicio de sobreajuste. De igual manera, las curvas de entrenamiento y validación permanecen cercanas.

Desempeño en el Conjunto de Prueba

De las 2,250 imágenes de prueba, que el modelo no vio durante el entrenamiento, se clasificaron correctamente 2,239 imágenes. Como resultado, la precision fue de 99.51% con una pérdida de 0.0165 con 11 errores.

La siguiente figura muestra la Matriz de Confusión con el conjunto de datos de prueba usando el modelo a color. Se utilizaron 450 imágenes por clase:



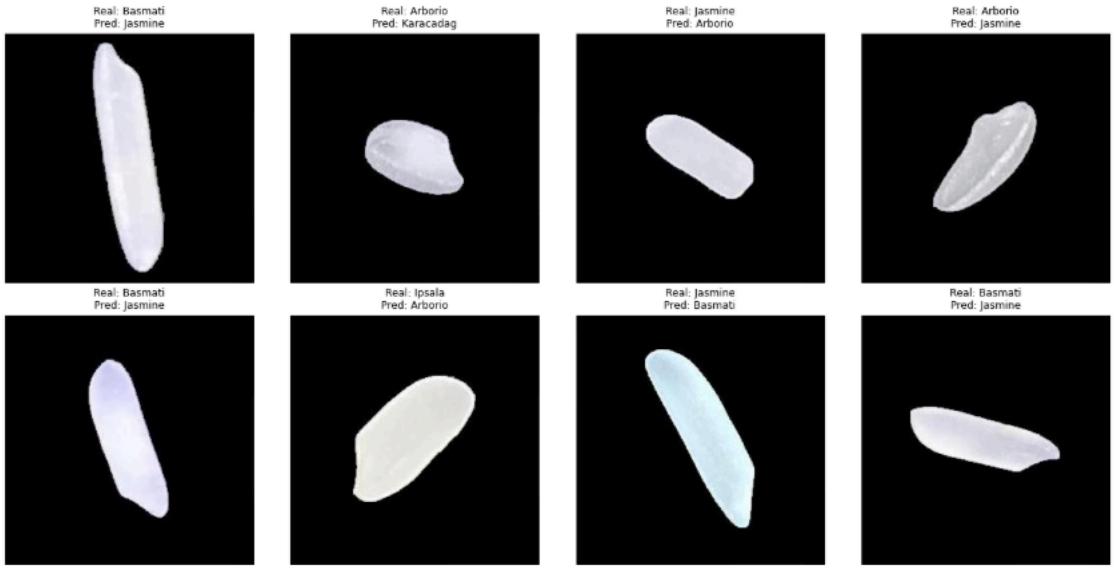
La siguiente tabla muestra la precisión, exhaustividad y F1 por variedad en el conjunto de prueba (modelo a color). Todas estas métricas por clase son presentadas por el reporte de clasificación del modelo:

Variedad	Precisión	Exhaustividad (Recall)	F1
Arborio	0.9955	0.9911	0.9933
Basmati	0.9978	0.9911	0.9944
Ipsala	1.0000	0.9978	0.9989
Jasmine	0.9890	0.9956	0.9922
Karacadag	0.9934	1.0000	0.9967
Promedio macro	0.9951	0.9951	0.9951

Análisis de Errores

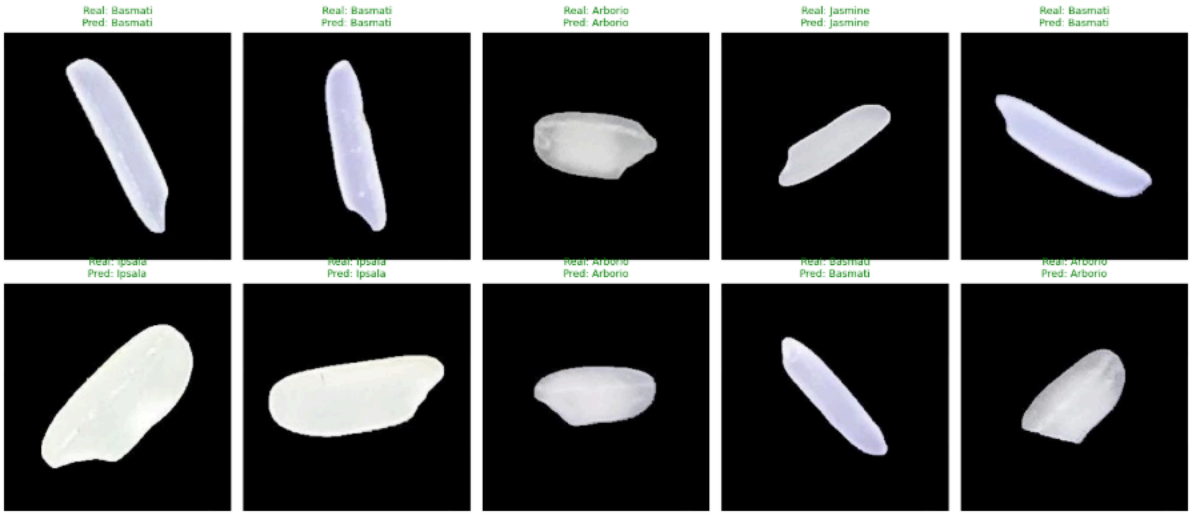
Los 11 errores se reparten en pocas confusiones. La más frecuente es Basmati que se predice como Jasmine (4 granos - 36% de los errores), seguida de Arborio confundida por Karacadag (3 granos). Los demás errores son aislados, de un grano cada uno. Según la matriz de confusión, estos son los errores: Arborio como Jasmine, Ipsala como Arborio, Jasmine como Arborio y Jasmine como Basmati. Una de las explicaciones por las cuales Jasmine tiene la precisión más baja con un valor de 0.9890, es porque 5 de los 11 errores se deben a que el modelo clasifica erróneamente otros granos de arroz como Jasmine. Ningún grano de Karacadag se clasificó mal, por eso su exhaustividad es de 1.000. Además, ningún grano de otra variedad se clasificó como Ipsala.

La siguiente figura muestra los ejemplos de granos mal clasificados en el conjunto de prueba (8 de los 11 errores).



Los granos de Basmati confundidos con Jasmine comparten similitudes físicas, son alargados, lo que explica su confusión. En general, los errores corresponden a granos de forma similar entre dos variedades o con una orientación atípica.

La siguiente figura muestra las predicciones sobre una muestra aleatoria de 10 imagenes de prueba. El texto verde significa acierto en la predicción:



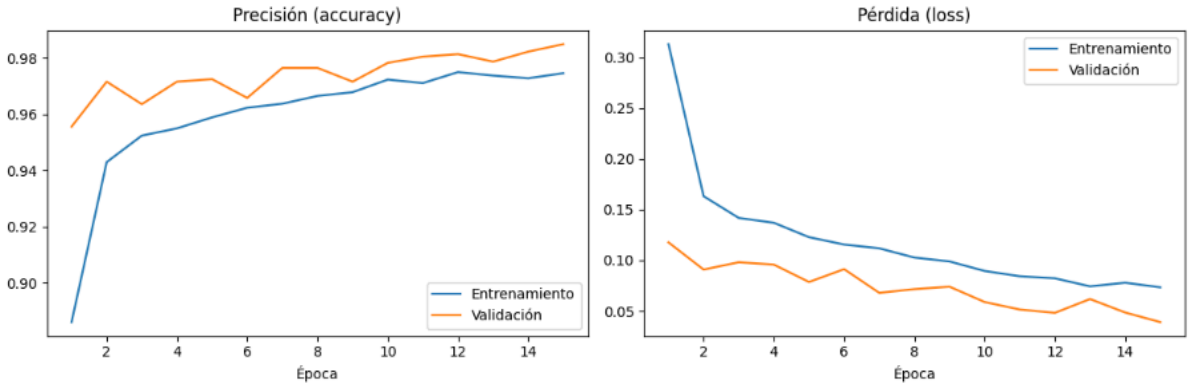
Las 10 imágenes de esta muestra se clasificaron bien. Este resultado es esperado, porque la tasa de error es de 0.49% (11 granos de 2,250). Esta figura ilustra el funcionamiento del modelo, pero la evidencia concreta de su desempeño se encuentra en la matriz de confusión y en las métricas por clase.

Experimento: Escala de Grises

Para medir cuánto aporta el color, se repitió el entrenamiento completo con GRAYSCALE = TRUE, pero sin hacer más cambios. Se utilizó el mismo subconjunto y división de datos, misma arquitectura, mismos hiperparámetros y misma semilla. La única diferencia es que cada imagen entra con 1 canal (gris) en lugar de 3. Por eso la primera capa convolucional pasa de 896 a 320 parámetros y el modelo de 4,288,325 a 4,287,749. Se verificó que los lotes tienen forma (32, 128, 128, 1) y valores entre 0 y 1.

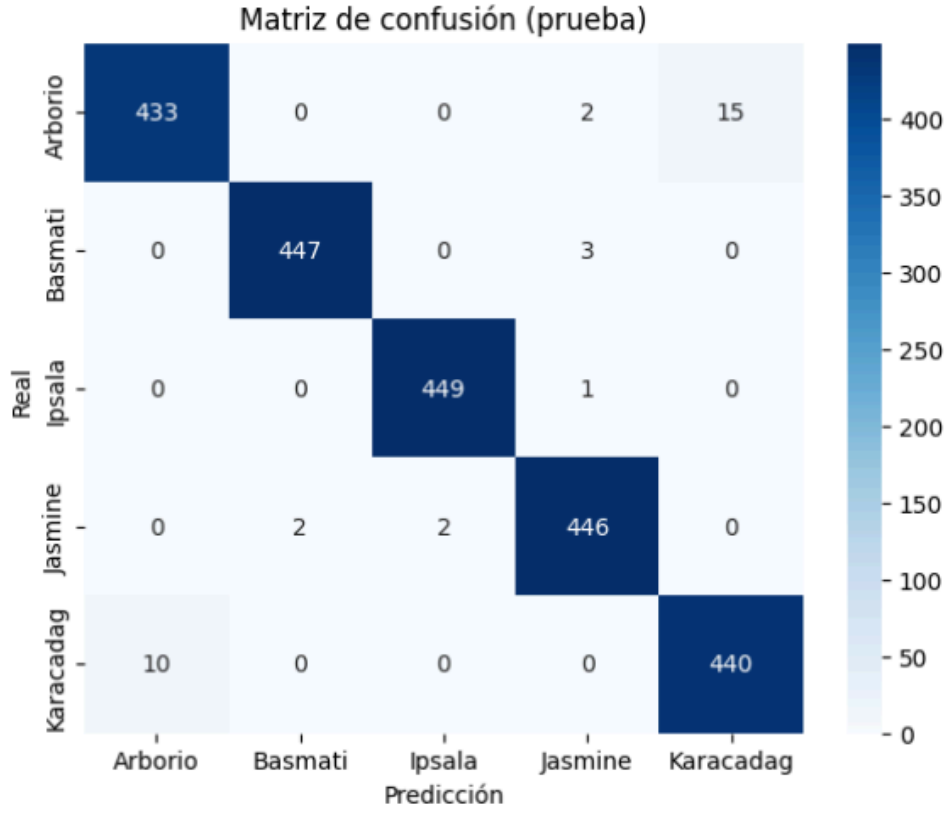
A diferencia de la corrida a color, el entrenamiento llegó a las 15 épocas sin que EarlyStopping lo detuviera. La pérdida de validación seguía bajando en la última época, por ende el límite de 15 épocas limitó el entrenamiento.

La siguiente figura muestra la precisión y pérdida por época en el entrenamiento en escala de grises:



La precisión de entrenamiento sube de 0.886 a 0.975 y la de validación de 0.956 a 0.985. La pérdida de validación baja de 0.118 a 0.039. Las curvas son más suaves que en el experimento a color, que mostró oscilaciones en las épocas 6 y 12. La validación se mantiene por encima del entrenamiento, debido a que el Dropout y el aumento de datos solamente se activan al entrenar. En el conjunto de prueba, el modelo en escala de grises se obtuvo una **precisión de 98.44%** (2,215 de 2,250 imágenes) y una pérdida de 0.0389 con 35 errores.

La siguiente figura muestra la matriz de confusión en la prueba del modelo usando las imágenes en escala de grises:



La siguiente tabla muestra las métricas por variedad en la prueba del modelo usando las imágenes en escala de grises:

Variedad	Precisión	Exhaustividad (Recall)	F1
Arborio	0.9774	0.9622	0.9698
Basmati	0.9955	0.9933	0.9944
Ipsala	0.9956	0.9978	0.9967
Jasmine	0.9867	0.9911	0.9889
Karacadag	0.9670	0.9778	0.9724
Promedio Macro	0.9845	0.9844	0.9844

Comparación entre Color y Escala de Grises

La siguiente tabla muestra la comparación de los dos entrenamientos:

Métrica	Color	Escala de Grises
Canales de Entrada	3	1
Parámetros	4,288,325	4,287,749
Épocas Ejecutadas	12 (mejor: 8)	15 (mejor: 15)
Mejor Pérdida de Validación	0.0109	0.0390
Precisión de Validación (Mejor Época)	0.9960	0.9849
Pérdida en Prueba	0.0165	0.0389
Precisión en Prueba	99.51%	98.44%
Errores en Prueba	11 de 2,250	35 de 2,250
F1 Promedio Macro	0.9951	0.9844

La siguiente tabla muestra el cambio en el F1 por variedad al quitar el color:

Variedad	F1 Color	F1 Grises	Diferencia
Arborio	0.9933	0.9698	-0.0235
Basmati	0.9944	0.9944	0.0000
Ipsala	0.9989	0.9967	-0.0022
Jasmine	0.9922	0.9889	-0.0033
Karacadag	0.9967	0.9724	-0.0243

Quitar el color costó 1.07 puntos porcentuales de precisión (99.51% a 98.44%) lo que equivale a 24 imágenes más mal clasificadas. El modelo sigue siendo muy bueno sin color, lo que significa que la mayoría de la información se encuentra en la forma y la textura del grano. Sin embargo, el efecto no es uniforme. De los 24 errores adicionales, 23 corresponden a granos de Arborio y Karacadag (4 errores a color comparado a 27 errores en grises). Además, 25 de los 35 errores en grises (71%) son causados porque el modelo confunde entre sí dos variedades: Arborio y Karacadag. Cuando se utilizó el color, solamente hubo 3 de ese tipo de errores de los 11 errores. Las otras tres variedades casi no cambian (los valores de F1 son entre 0.0000 y -0.0033).

La siguiente figura muestra los granos mal clasificados por el modelo en escala de grises (las imágenes que se muestran son las originales a color, pero el modelo recibió las imágenes en gris):



Los errores de la figura son en su mayoría granos cortos y redondeados de forma muy parecida entre Arborio y Karacadag. Una interpretación es que cuando la forma es casi igual, el tono (color) ayuda a separarlas. Por eso cuando se quita el color, el modelo pierde esa información y

clasifica el grano de arroz de forma equivocada. Es una interpretación personal y no está comprobada, porque el experimento no aísla qué rasgo del color es el que importa. Me gustaría aclarar que en las imágenes de muestra, los granos de Basmati parecen tener un ligero tono lavanda comparado al blanco de las otras variedades de granos de arroz. Mi interpretación es que esto sugería que el modelo podría apoyarse en ese tono y no en la forma del grano. La hipótesis no se sostuvo, porque el F1 de Basmati es el mismo con y sin color (0.9944). Con estos resultados se puede concluir que el modelo reconoce el grano Basmati por su forma alargada y no por el tono lavanda.

Variabilidad entre Corridas

Antes de hacer el experimento con las imágenes a color presentado en este reporte, se hizo una primera ejecución del código con las imágenes a color con exactamente la misma configuración. Esta primera ejecución se detuvo en la época 14, porque su mejor época fue la 10, y obtuvo 99.11% de precisión en prueba con 20 errores en su clasificación de imágenes. En la segunda ejecución del código, que es el experimento presentado en este reporte, tuvo 99.51% de precisión en prueba y 11 errores. Esta diferencia de 0.40 puntos entre dos entrenamientos idénticos muestra cuánto afecta en el resultado la aleatoriedad del entrenamiento en su inicialización, aumento de datos y orden de los lotes en el GPU.

Para saber si la diferencia entre color y grises es un efecto real o solamente azar, se compara con la diferencia entre las dos ejecuciones a color que fue de 0.40 puntos. La brecha entre escala de grises y color (1.07 puntos) es claramente mayor que ese valor. La brecha entre grises y la primera ejecución también es mayor, porque es de 0.67 puntos. Con solamente dos ejecuciones a color y una en escala de grises no se puede constituir una prueba estadística. Además, el modelo en escala de grises no llegó a converger en 15 épocas, así que con más entrenamiento la brecha podría reducirse. Lo que sí es consistente en todas las comparaciones es que el efecto se concentra en Arborio y Karacadag.

7. Conclusiones

La CNN propuesta resuelve la clasificación de las cinco variedades de arroz con 99.51% de precisión en el conjunto de prueba con valores de precisión, exhaustividad y F1 superiores a 0.98 en todas las clases. Esto cumple el objetivo de clasificación del proyecto y muestra que una arquitectura relativamente sencilla con tres bloques convolucionales es suficiente cuando las clases son visualmente separables y los datos están bien preparados en la fase del preprocesamiento.

Al repetir el entrenamiento en escala de grises se obtuvo 98.44% de precisión. El color aporta una mejora modesta (1.07 puntos) concentrada en separar Arborio de Karacadag, mientras que las otras variedades se distinguen solamente por su forma.

Lo que funcionó:

- La partición estratificada en tres conjuntos, con los datos de prueba reservados para usarlos al final, hace creíble la cifra reportada. Las curvas de entrenamiento y validación permanecen cercanas sin señales claras de sobreajuste.
- El preprocesamiento (tamaño uniforme, escalado al rango de [0,1] y etiquetas one-hot encoding), el aumento de datos y el Dropout produjeron un entrenamiento estable. EarlyStopping y la restauración de los mejores pesos evitaron entrenar de más y terminar con sobreajuste en el modelo.

Limitaciones:

- Los datos son de un laboratorio. Cada imagen contiene un grano aislado sobre un fondo negro, iluminación controlada y una sola fuente de captura. La precisión de 99% no garantiza el mismo desempeño con fotografías de granos mezclados, otros fondos o cámaras distintas.
- Se usó un subconjunto de 3,000 imágenes por clase (20% del dataset). Con todos los datos podría mejorar, aunque con un rendimiento ya tan alto el margen de mejora es pequeño.
- Las dos ejecución del código con las imágenes a color dieron 99.11% y 99.51% lo que muestra que la aleatoriedad del entrenamiento cambia el resultado unas décimas de punto. El experimento en escala de grises se entrenó una sola vez, así que las diferencias pequeñas entre variedades no son concluyentes.
- Las oscilaciones de la validación (época 6 y 12 en la corrida a color) sugieren que la tasa de aprendizaje constante de 0.001 podría ser algo alta. No se probó ningún otro valor para la tasa de aprendizaje.

Aprendizaje y Trabajo Futuro:

- Entrenar el modelo en escala de grises con más épocas, porque solamente se utilizó 15 épocas y la pérdida de validación seguía bajando. También se puede repetir cada configuración con varias semillas para saber si la diferencia de aproximadamente 1 punto es consistente.
- Reducir la tasa de aprendizaje durante el entrenamiento para suavizar las oscilaciones de validación, por ejemplo con ReduceLROnPlateau.
- Evaluar la robustez con imágenes tomadas fuera del dataset (otros fondos, granos juntos, otra iluminación) y comparar con Transfer Learning (MobileNetV2).
- Revisar siempre la matriz de confusión y el F1 por clase, en lugar de solamente utilizar la precisión global como métrica absoluta. La precisión es suficiente para medir con clases balanceadas, pero las otras métricas muestran dónde se concentra el error, en este caso Jasmine a color y Arborio-Karacadag en grises).

Implicaciones Éticas

Esta problemática no involucra datos personales, pero tendría consecuencias en un uso comercial. Por ejemplo, un error de clasificación podría etiquetar de forma incorrecta un producto de mayor precio y eso puede tener un impacto económico para productores y consumidores. Por eso el modelo debería usarse como apoyo a una revisión humana y

validarse con datos reales de cualquier aplicación industrial. Finalmente, se debe documentar la fuente de los datos y en las condiciones con las que fueron capturados.

8. Referencias

Koklu, M., Cinar, I., y Taspinar, Y. S. (2021). Classification of rice varieties with deep learning methods. Computers and Electronics in Agriculture, 187, 106285.

Koklu, M. (s.f.). Rice image dataset [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/muratkokludataset/rice-image-dataset

TensorFlow. (s.f.). Clasificación de imágenes. https://www.tensorflow.org/tutorials/images/classification